

GN Quick Start guide

GN Quick Start guide

- Running GN
- Setting up a build
- Passing build arguments
- Cross-compiling to a target OS or architecture
- Step-by-step
 - Adding a build file
 - Testing your addition
 - Declaring dependencies
 - Test the binary
 - Putting settings in a config
 - Default configs
- Add a new build argument
- Don't know what's going on?
 - The "desc" command

Running GN

You just run `gn` from the command line. For large projects, GN is versioned and distributed with the source checkout.

- For Chromium and Chromium-based projects, there is a script in `depot_tools`, which is presumably in your PATH, with this name. The script will find the binary in the source tree containing the current directory and run it.
- For Fuchsia in-tree development, run `fx gn ...` which will find the right GN binary and run it with the given arguments.
- For other projects, see your project's documentation.

Setting up a build

Unlike some other build systems, with GN you set up your own build directories with the settings you want. This lets you maintain as many different builds in parallel as you need.

Once you set up a build directory, the Ninja files will be automatically regenerated if they're out of date when you build in that directory so you should not have to re-run GN.

To make a build directory:

```
gn gen out/my_build
```

Passing build arguments

Set build arguments on your build directory by running:

```
gn args out/my_build
```

This will bring up an editor. Type build args into that file like this:

```
is_component_build = true
is_debug = false
```

The available variables will depend on your build (this example is from Chromium). You can see the list of available arguments and their default values by typing

```
gn args --list out/my_build
```

on the command line. Note that you have to specify the build directory for this command because the available arguments can change according to the build.

Chrome developers can also read the [Chrome-specific build configuration](#) instructions for more information.

Cross-compiling to a target OS or architecture

Run `gn args out/Default` (substituting your build directory as needed) and add one or more of the following lines for common cross-compiling options.

```
target_os = "chromeos"
target_os = "android"

target_cpu = "arm"
target_cpu = "x86"
target_cpu = "x64"
```

See [GN cross compiles](#) for more info.

Step-by-step

Adding a build file

Go to the directory `examples/simple_build`. This is the root of a minimal GN repository.

In that directory there is a `tutorial` directory. There is already a `tutorial.cc` file that's not hooked up to the build. Create a new `BUILD.gn` file in that directory for our new target.

```
executable("tutorial") {
  sources = [
    "tutorial.cc",
  ]
}
```

Now we just need to tell the build about this new target. Open the `BUILD.gn` file in the parent (`simple_build`) directory. GN starts by loading this root file, and then loads all dependencies outward from here, so we just need to add a reference to our new target from this file.

You could add our new target as a dependency from one of the existing targets in the `simple_build/BUILD.gn` file, but it usually doesn't make a lot of sense to have an executable as a dependency of another executable (they can't be linked). So let's make a "tools" group. In GN, a "group" is just a collection of dependencies that's not compiled or linked:

```
group("tools") {
  deps = [
    # This will expand to the name "//tutorial:tutorial" which is the full name
    # of our new target. Run "gn help labels" for more.
    "//tutorial",
  ]
}
```

Testing your addition

From the command line in the `simple_build` directory:

```
gn gen out
ninja -C out tutorial
out/tutorial
```

You should see "Hello, world." output to the console.

Side note: GN encourages target names for static libraries that aren't globally unique. To build one of these, you can pass the label with its path (but no leading `//`) to ninja:

```
ninja -C out some/path/to/target:my_target
```

Declaring dependencies

Let's look at the targets defined in [examples/simple_build/BUILD.gn](#). There is a static library that defines one function, `GetStaticText()`:

```
static_library("hello_static") {
  sources = [
    "hello_static.cc",
    "hello_static.h",
  ]
}
```

There is also a shared library that defines one function `GetSharedText()`:

```
shared_library("hello_shared") {  
  sources = [  
    "hello_shared.cc",  
    "hello_shared.h",  
  ]  
  
  defines = [ "HELLO_SHARED_IMPLEMENTATION" ]  
}
```

This also illustrates how to set preprocessor defines for a target. To set more than one or to assign values, use this form:

```
defines = [  
  "HELLO_SHARED_IMPLEMENTATION",  
  "ENABLE_DOOM_MELON=0",  
]
```

Now let's look at the executable that depends on these two libraries:

```
executable("hello") {  
  sources = [  
    "hello.cc",  
  ]  
  
  deps = [  
    ":hello_shared",  
    ":hello_static",  
  ]  
}
```

This executable includes one source file and depends on the previous two libraries. Labels starting with a colon refer to a target with that name in the current BUILD.gn file.

Test the binary

From the command line in the `simple_build` directory:

```
ninja -C out hello  
out/hello
```

Note that you **didn't** need to re-run GN. GN will automatically rebuild the ninja files when any build file has changed. You know this happens when ninja prints `[1/1] Regenerating ninja files` at the beginning of execution.

Putting settings in a config

Users of a library often need compiler flags, defines, and include directories applied to them. To do this, put all such settings into a "config" which is a named collection of settings (but not sources or dependencies):

```
config("my_lib_config") {
  defines = [ "ENABLE_DOOM_MELON" ]
  include_dirs = [ "../third_party/something" ]
}
```

To apply a config's settings to a target, add it to the `configs` list:

```
static_library("hello_shared") {
  ...
  # Note "+=" here is usually required, see "default configs" below.
  configs += [
    ":my_lib_config",
  ]
}
```

A config can be applied to all targets that depend on the current one by putting its label in the `public_configs` list:

```
static_library("hello_shared") {
  ...
  public_configs = [
    ":my_lib_config",
  ]
}
```

The `public_configs` also applies to the current target, so there's no need to list a config in both places.

Default configs

The build configuration will set up some settings that apply to every target by default. These will normally be set as a default list of configs. You can see this using the "print" command which is useful for debugging:

```
executable("hello") {
  print(configs)
}
```

Running GN will print something like:

```
$ gn gen out
[ "../build:compiler_defaults", "../build:executable_ldconfig" ]
Done. Made 5 targets from 5 files in 9ms
```

Targets can modify this list to change their defaults. For example, the build setup might turn off exceptions by default by adding a `no_exceptions` config, but a target might re-enable them by replacing it with a different one:

```
executable("hello") {
    ...
    configs -= [ "//build:no_exceptions" ] # Remove global default.
    configs += [ "//build:exceptions" ] # Replace with a different one.
}
```

Our print command from above could also be expressed using string interpolation. This is a way to convert values to strings. It uses the symbol "\$" to refer to a variable:

```
print("The configs for the target $target_name are $configs")
```

Add a new build argument

You declare which arguments you accept and specify default values via `declare_args`.

```
declare_args() {
    enable_teleporter = true
    enable_doom_melon = false
}
```

See `gn help buildargs` for an overview of how this works.

See `gn help declare_args` for specifics on declaring them.

It is an error to declare a given argument more than once in a given scope, so care should be used in scoping and naming arguments.

Don't know what's going on?

You can run GN in verbose mode to see lots of messages about what it's doing. Use `-v` for this.

The "desc" command

You can run `gn desc <build_dir> <targetname>` to get information about a given target:

```
gn desc out/Default //foo/bar:say_hello
```

will print out lots of exciting information. You can also print just one section. Lets say you wanted to know where your `TWO_PEOPLE` define came from on the `say_hello` target:

```
> gn desc out/Default //foo/bar:say_hello defines --blame
...lots of other stuff omitted...
From //foo/bar:hello_config
    (Added by //foo/bar/BUILD.gn:12)
    TWO_PEOPLE
```

Another particularly interesting variation:

```
gn desc out/Default //base:base_i18n deps --tree
```

See `gn help desc` for more.